NAME : SAFIYA B

REG NO: 192411043

CONSTRAINT BASED PROBLEM SOLVING

Boolean Circuit Satisfiability

Problem: Determine whether there exists an assignment of Boolean inputs that makes the circuit output True.

Example circuit:

$$F = (A \wedge B) \vee (\neg C)$$

The program tries input combinations, evaluates the gates, and stops as soon as it finds a satisfying assignment.

```
# Boolean Circuit Satisfiability
# Backtracking with pruning

class BooleanCircuit:
    def __init__(self):
        self.variables = ['A', 'B', 'C']

    # Evaluate the Boolean circuit
    # F = (A AND B) OR (NOT C)
    def evaluate(self, assignment):
        A = assignment.get('A')
        B = assignment.get('B')
        C = assignment.get('C')

        # AND gate
        if A is not None and B is not None:
            and_result = A and B
        else:
            and_result = None

        # NOT gate
        if C is not None:
            not_result = not C
        else:
            not_result = None

        # OR gate
        if and_result is True or not_result is Tr
            return True

        if and_result is False and not_result is
            return False

        return None
```

```
Console

Run started
Initializing environment
Installing packages
Running code
Boolean Circuit: F = (A AND B) OR (NOT C)
------------------------------------------
Result: SATISFIABLE
Satisfying assignment:
A = 0
B = 0
C = 0
Circuit Output = 1
Run completed in 6072.899999976158ms
```

So the circuit is **satisfiable**.

Constraints of Logical Gates

Gate	Constraint
AND	Output is 1 only when all inputs are 1
OR	Output is 1 when at least one input is 1
NOT	Output is opposite of its input
XOR	Output is 1 when inputs are different
NAND	Output is 0 only when all inputs are 1
NOR	Output is 1 only when all inputs are 0

Constraint Propagation and Pruning

The important optimization is:

Partial assignment → Evaluate circuit

↓

Output = FALSE?

↓ Yes

Prune branch

For example, if a partial assignment already proves that the circuit can never produce True, the program **does not continue checking the remaining combinations**.

Without pruning, n Boolean inputs require up to:

$$2^n$$

assignments.

With pruning, many branches can be eliminated earlier.

Correctness

The algorithm is correct because:

1. It assigns every input variable either 0 or 1.
2. Every possible assignment is considered unless its branch is safely pruned.
3. A branch is pruned only when the circuit is already proven to be False.
4. If the algorithm returns an assignment, evaluating the circuit gives True.
5. If no assignment is found, no satisfying assignment exists.

Complexity

For n input variables:

- **Worst-case time:** O(2^n × G), where G is the number of gates.
- **Space:** O(n) for the recursion and assignments.
- **With pruning:** practical execution can be much faster, depending on circuit structure.

Why Boolean Circuit SAT is NP-Complete

Circuit-SAT is in NP because, given an input assignment, we can evaluate all gates in polynomial time and verify whether the final output is True.

It is **NP-hard** because other NP problems can be transformed into Boolean circuit satisfiability in polynomial time.

Therefore:

Boolean Circuit SAT is NP-Complete

Conclusion

The implemented backtracking algorithm successfully determines whether a Boolean circuit has a satisfying input assignment. **Constraint propagation and pruning** reduce unnecessary searches, although the worst-case exponential complexity remains. This demonstrates why Circuit-SAT is computationally difficult for large numbers of inputs and why it is an important **NP-Complete problem**.